INTERPOLAÇÃO

- Em engenharia às vezes é necessário estimar valores intermediários entre pontos em que o valor da função é conhecido
- O método mais comum usado é a interpolação polimonial
- A equação de um polinômio de grau n é dada por:

$$f(x) = a_0 + a_1 x + a_2 x^2 + a_3 x^3 + \cdots + a_n x^n$$

- Para um conjunto de n+1 pontos existe apenas um polinômio de grau n que passa por todos esses pontos

INTERPOLAÇÃO LINEAR

- A forma mais fácil de interpolação é conectar dois pontos conhecidos usando uma linha reta.
- Partindo dos pontos conhecidos
  - Em $x_0 \Rightarrow f(x_0)$
  - Em $x_1 \Rightarrow f(x_1)$
- Para estimar o valor da função em um ponto x situado entre x_0 e x_1, pode-se partir de:

$$\frac{f_1(x) - f(x_0)}{x - x_0} = \frac{f(x_1) - f(x_0)}{x_1 - x_0}$$

em que $f_1(x)$ é a estimativa de f(x) que se quer obter

- Isolando-se $f_1(x)$ tem-se:

$$f_1(x) = f(x_0) + \frac{f(x_1) - f(x_0)}{x_1 - x_0}(x - x_0)$$

APLICAÇÃO USANDO INTERPOLAÇÃO LINEAR

- A corrente em um fio foi medida com grande precisão como função do tempo, e obteve-se

t	0	0,1250	0,2500	0,3750	0,5000
i	0	6,2402	7,7880	4,8599	0,0000

Desenvolva um programa para realizar a interpolação linear e estime o valor da corrente elétrica para t = 0,22.

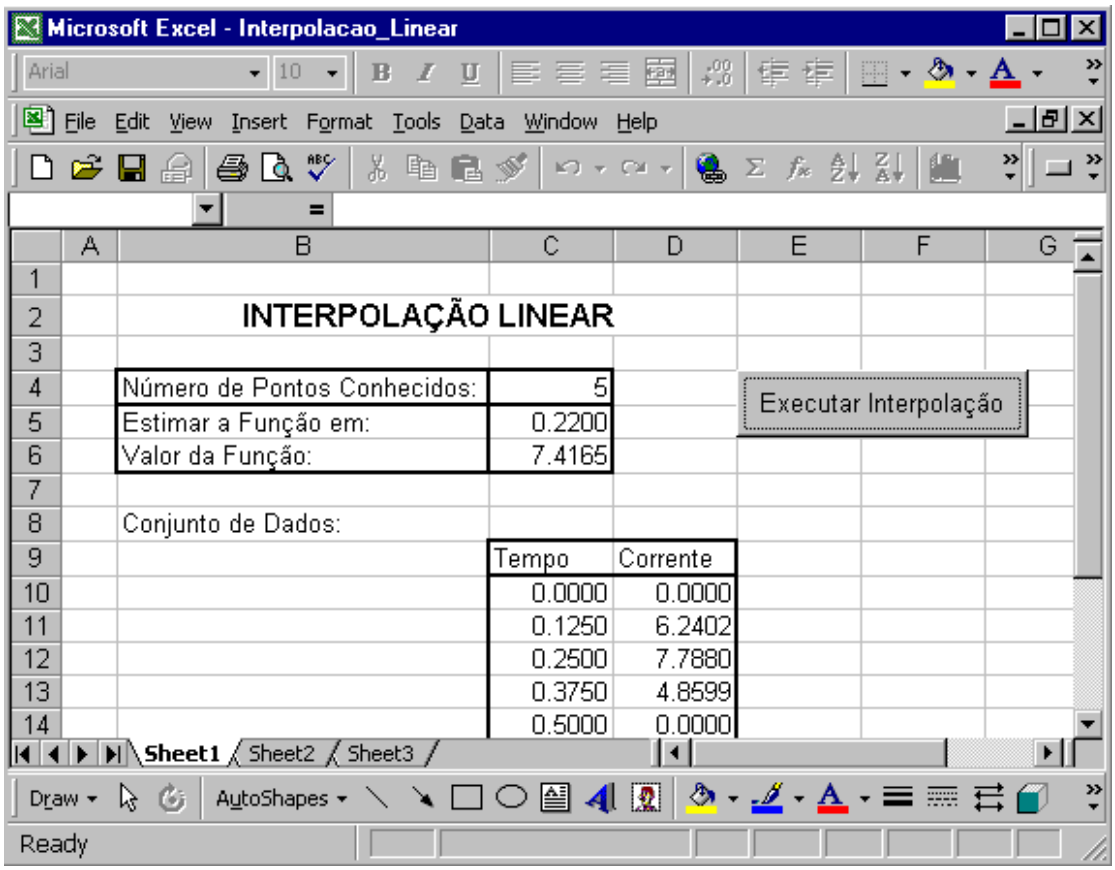
- Uma subrotina para fazer interpolação linear pode ser escrita da seguinte forma:

```
SUBROUTINE LinInt(x,y,n,xi,yint)
LOCAL i, flag
flag=0
DO i=1,n
```

```
  IF ((x(i) > xi) and (flag = 0))
    yint=y(i-1)+(y(i)-y(i-1))*(xi-x(i-1))/(x(i)-x(i-1))
     flag=1
  END IF
END DO
END LinInt
```

Implemente essa subrotina usando a linguagem VBA existente no excel e resolva o problema proposto.

- Usando a planilha excel do tipo:



- As subrotinas implementadas usando o VBA podem ser escritas assim:

```
Private Sub CommandButton1_Click()
  Call Entrada_de_Dados1
End Sub
Private Sub Entrada_de_Dados1()
  Dim n As Integer
  Dim x(100) As Double
  Dim y(100) As Double
  Dim xi As Double
```

```
  Dim yint As Double
  Dim ilin As Integer
  Dim icol As Integer
  n = Sheet1.Cells(4, 3)
  xi = Sheet1.Cells(5, 3)
  For ilin = 10 To 10 + n - 1
    x(ilin - 10) = Sheet1.Cells(ilin, 3)
    y(ilin - 10) = Sheet1.Cells(ilin, 4)
  Next ilin
  Call LinInt(x, y, n, xi, yint)
  Sheet1.Cells(6, 3) = yint
End Sub
Private Sub LinInt(x, y, n, xi, yint)
Dim i As Integer
Dim flag As Integer
flag = 0
For i = 1 To n
  If ((x(i) > xi) And (flag = 0)) Then
    yint = y(i - 1) + (y(i) - y(i - 1)) * (xi - x(i - 1)) / (x(i) - x(i - 1))
    flag = 1
  End If
Next i
End Sub
```

INTERPOLAÇÃO QUADRÁTICA

- Caso três pontos estejam disponíveis um polinômio de segunda ordem pode ser usado
- Um polinômio de segunda ordem pode ser escrito na forma:

$$f_2(x) = b_0 + b_1(x - x_0) + b_2(x - x_0)(x - x_1)$$

- Essa equação pode ser desenvolvido e re-escrita na forma:

$$f_2(x) = b_0 + b_1 x - b_1 x_0 + b_2 x^2 + b_2 x_0 x_1 - b_2 x x_0 - b_2 x x_1$$

- Reagrupando os termos:

$$f_2(x) = a_0 + a_1 x + a_2 x^2$$

em que

$$a_0 = b_0 - b_1 x_0 + b_2 x_0 x_1$$
$$a_1 = b_1 - b_2 x_0 - b_2 x_1$$
$$a_2 = b_2$$

- Substituindo $x = x_0$ na equação do polinômio:

$$b_0 = f(x_0)$$

- Substituindo $x = x_1$ e o valor de b_0 na equação do polinômio:

$$b_1 = \frac{f(x_1) - f(x_0)}{x_1 - x_0}$$

- Substituindo $x = x_2$ e os valores de b_0 e b_1 na equação do polinômio:

$$b_2 = \frac{\dfrac{f(x_2) - f(x_1)}{x_2 - x_1} - \dfrac{f(x_1) - f(x_0)}{x_1 - x_0}}{x_2 - x_0}$$

APLICAÇÃO USANDO INTERPOLAÇÃO QUADRÁTICA

- A corrente em um fio foi medida com grande precisão como função do tempo, e obteve-se

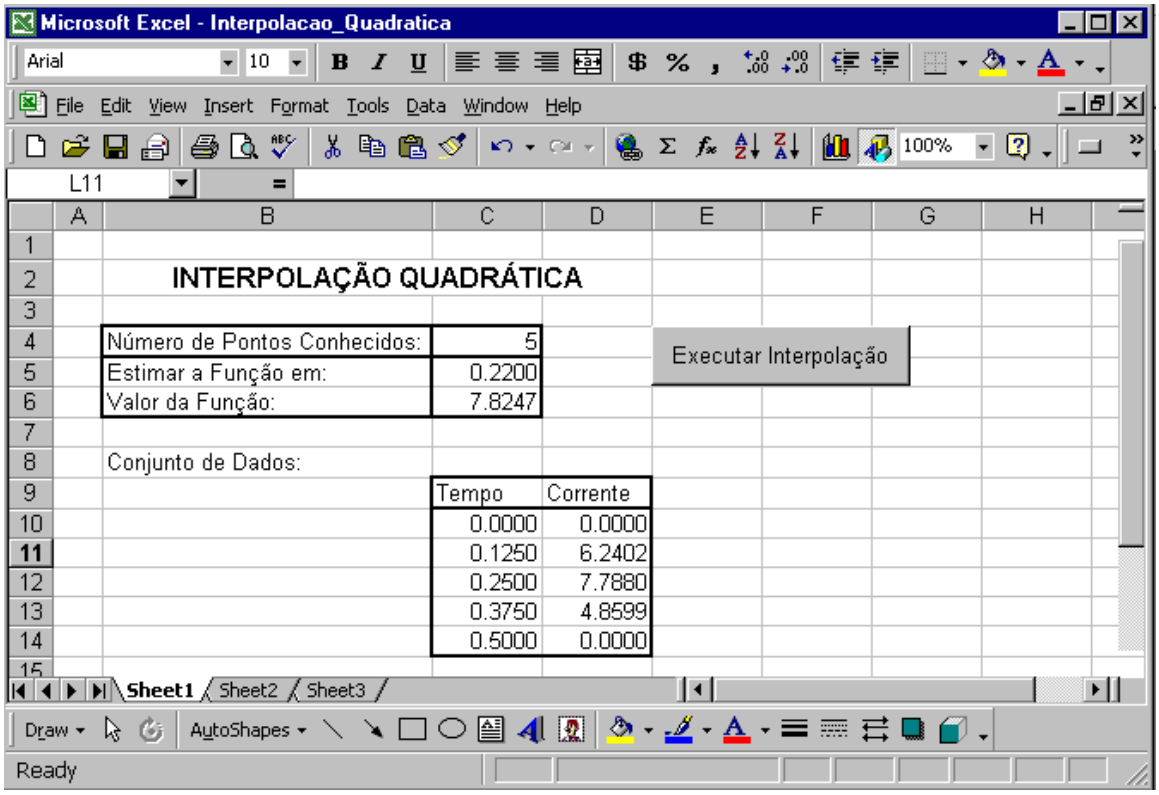
T	0	0,1250	0,2500	0,3750	0,5000
I	0	6,2402	7,7880	4,8599	0,0000

Desenvolva um programa para realizar a interpolação quadrática e estime o valor da corrente elétrica para t = 0,22.

- Uma subrotina para fazer interpolação quadrática pode ser escrita da seguinte forma:

```
SUBROUTINE QuaInt(x,y,n,xi,yint)
LOCAL i, flag, a0, a1, a2, b0, b1, b2
flag=0
DO i=1,n
  IF ((x(i) > xi) and (flag = 0))
    b0=y(i-1)
    b1=(y(i)-y(i-1))/(x(i)-x(i-1))
    b2=((y(i+1)-y(i))/(x(i+1)-x(i)-b1)
    a0 = b0 - b1*x(i-1) + b2*x(i-1)*x(i)
    a1= b1 - b2*x(i-1) - b2*x(i)
    a3=b2
    yint = a0 + a1*xi + a2*xi^2
     flag=1
  END IF
END DO
END QuaInt
```

- Usando a planilha excel do tipo:



- As subrotinas implementadas usando o VBA podem ser escritas assim:

```
Private Sub CommandButton1_Click()
  Call Entrada_de_Dados2
End Sub
Private Sub Entrada_de_Dados2()
  Dim n As Integer
  Dim x(100) As Double
  Dim y(100) As Double
  Dim xi As Double
  Dim yint As Double
  Dim ilin As Integer
  Dim icol As Integer
  n = Sheet1.Cells(4, 3)
  xi = Sheet1.Cells(5, 3)
  For ilin = 10 To 10 + n - 1
    x(ilin - 10) = Sheet1.Cells(ilin, 3)
    y(ilin - 10) = Sheet1.Cells(ilin, 4)
  Next ilin
  Call QuaInt(x, y, n, xi, yint)
  Sheet1.Cells(6, 3) = yint
End Sub
Private Sub QuaInt(x, y, n, xi, yint)
```

```
Dim i As Integer
Dim flag As Integer
Dim a0 As Double
Dim a1 As Double
Dim a3 As Double
Dim b0 As Double
Dim b1 As Double
Dim b2 As Double
flag = 0
For i = 1 To n
  If ((x(i) > xi) And (flag = 0)) Then
    b0 = y(i - 1)
    b1 = (y(i) - y(i - 1)) / (x(i) - x(i - 1))
    b2 = ((y(i + 1) - y(i)) / (x(i + 1) - x(i)) - b1) / (x(i + 1) - x(i - 1))
    a0 = b0 - b1 * x(i - 1) + b2 * x(i - 1) * x(i)
    a1 = b1 - b2 * x(i - 1) - b2 * x(i)
    a2 = b2
    yint = a0 + a1 * xi + a2 * xi * xi
    flag = 1
  End If
Next i
End Sub
```

FORMA GERAL DOS POLINÔMIOS DE INTERPOLAÇÃO DE NEWTON

- Um polinômio que passa por n+1 pares de dados de ordem n pode ser representado por

$$f_n(x) = b_0 + b_1(x - x_0) + b_2(x - x_0)(x - x_1) + \cdots + b_n(x - x_0)(x - x_1)\cdots(x - x_{n-1})$$

- Para avaliar os valores dos coeficientes do polinômio utiliza-se as relações

$$b_0 = f(x_0)$$

$$b_1 = f[x_1, x_0]$$

$$b_2 = f[x_2, x_1, x_0]$$

$$\vdots$$

$$b_n = f[x_n, x_{n-1}, \cdots, x_1, x_0]$$

- As funções apresentadas para calcular os coeficientes do polinômio são obtidas a partir das equações de diferenças divididas dadas por:

$$f[x_i, x_j] = \frac{f(x_i) - f(x_j)}{x_i - x_j}$$

$$f[x_i, x_j, x_k] = \frac{f[x_i, x_j] - f[x_j, x_k]}{x_i - x_k}$$

$$f[x_n, x_{n-1}, \cdots, x_1, x_0] = \frac{f[x_n, x_{n-1}, \cdots, x_1] - f[x_{n-1}, x_{n-2}, \cdots, x_0]}{x_n - x_0}$$

- Usando as equações acima, podo-se escrever o polinômio interpolador da seguinte forma:

$$f_n(x) = f(x_0) + (x - x_0) f[x_1, x_0] + (x - x_0)(x - x_1) f[x_2, x_1, x_0]$$
$$+ \cdots + (x - x_0)(x - x_1) \cdots (x - x_{n-1}) f[x_n, x_{n-1}, \cdots, x_0]$$

- O erro da interpolação pode ser estimado pela equação

$$R_n \cong f[x_{n+1}, x_n, x_{n-1}, \cdots, x_0](x - x_0)(x - x_1) \cdots (x - x_n)$$

ALGORITMO PARA O POLINÔMIO INTERPOLADOR DE NEWTON

- O seguinte algoritmo pode ser usado para se fazer interpolação:

```
SUBROUTINE NewInt(x,y,n,xi,yint,ea)
    LOCAL fdd_{n,n}
    LOCAL i,j,ordem
    LOCAL xterm,yint2
    DO i = 0,n
        Fdd_{i,0}=y_i
    END DO
    DO j = 1,n
        DO i = 0, n-j
            Fdd_{i,j}=(fdd_{i+1,j-1} – fdd_{i,j-1})/(x_{i+j}-x_i)
        END DO
    END DO
    xterm = 1
    yint_0 = fdd_{0,0}
    DO ordem = 1,n
        xterm = xterm*(xi-x_{ordem-1})
        yint2=yint_{ordem-1}+fdd_{0,ordem}*xterm
        ea_{ordem-1}=yint2-yint_{ordem-1}
        yint_{ordem}=yint2
    END ordem
END NewInt
```

- Nesse algoritmo os pares de pontos são transferidos nos vetores x e y, o número de pontos é transferido na variável n, o ponto para o qual se quer interpolar é transferido na variável xi, os resultados dos valores interpolados é apresentado no vetor yint, e o erro aproximado da interpolação é apresentado no vetor ea.

UNIVERSIDADE FEDERAL DE VIÇOSA
DEPARTAMENTO DE ENGENHARIA AGRÍCOLA
ENG590 - Métodos Numéricos Aplicados à Engenharia

APLICAÇÃO DO ALGORITMO PARA O POLINÔMIO INTERPOLADOR DE NEWTON

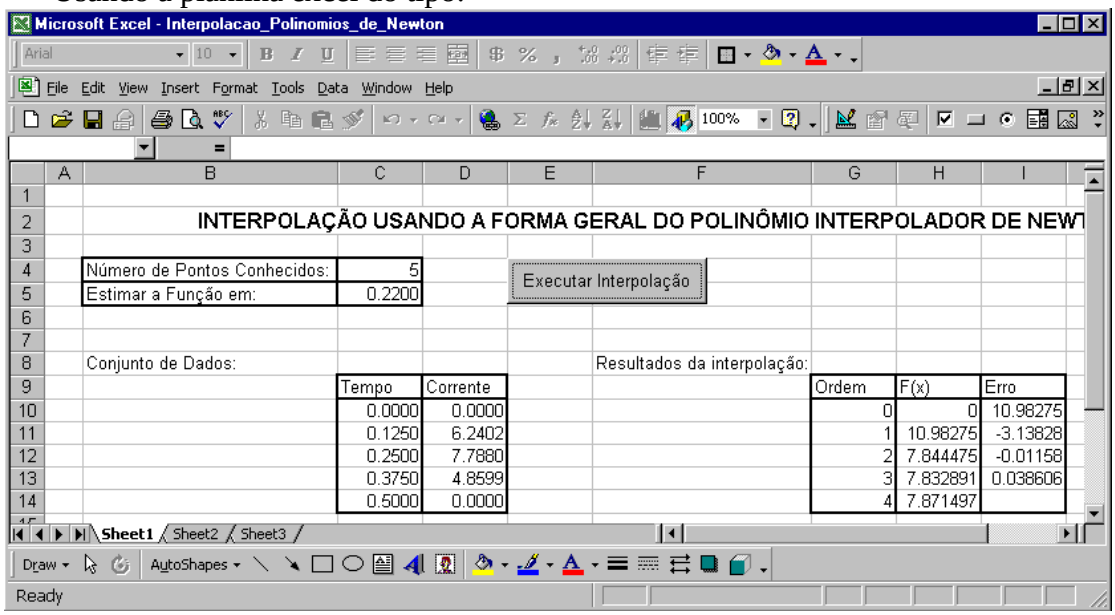
- A corrente em um fio foi medida com grande precisão como função do tempo, e obteve-se

T	0	0,1250	0,2500	0,3750	0,5000
I	0	6,2402	7,7880	4,8599	0,0000

Desenvolva um programa para realizar a interpolação quadrática e estime o valor da corrente elétrica para t = 0,22.

Prof. Daniel Marçal de Queiroz

- Usando a planilha excel do tipo:



- As subrotinas implementadas usando o VBA podem ser escritas assim:

```
Private Sub CommandButton1_Click()
  Call Entrada_de_Dados3
End Sub
Private Sub Entrada_de_Dados3()
  Dim n As Integer
  Dim x(100) As Double
  Dim y(100) As Double
  Dim yint(100) As Double
  Dim ea(100) As Double
  Dim xi As Double
  Dim ilin As Integer
  Dim icol As Integer
  n = Sheet1.Cells(4, 3) - 1
  xi = Sheet1.Cells(5, 3)
  For ilin = 10 To 10 + n
    x(ilin - 10) = Sheet1.Cells(ilin, 3)
    y(ilin - 10) = Sheet1.Cells(ilin, 4)
  Next ilin
  Call NewInt(x, y, n, xi, yint, ea)
  For ilin = 10 To 10 + n
     Sheet1.Cells(ilin, 7) = ilin - 10
     Sheet1.Cells(ilin, 8) = yint(ilin - 10)
     If ilin < (10 + n) Then
         Sheet1.Cells(ilin, 9) = ea(ilin - 10)
     End If
  Next ilin
```

```
End Sub
Private Sub NewInt(x, y, n, xi, yint, ea)
Dim i As Integer
Dim j As Integer
Dim ordem As Integer
Dim FDD(100, 100) As Double
Dim xterm As Double
Dim yint2 As Double
For i = 0 To n
  FDD(i, 0) = y(i)
Next i
For j = 1 To n
   For i = 0 To n - j
      FDD(i, j) = (FDD(i + 1, j - 1) - FDD(i, j - 1)) / (x(i + j) - x(i))
   Next i
Next j
xterm = 1
yint(0) = FDD(0, 0)
For Order = 1 To n
   xterm = xterm * (xi - x(Order - 1))
   yint2 = yint(Order - 1) + FDD(0, Order) * xterm
   ea(Order - 1) = yint2 - yint(Order - 1)
   yint(Order) = yint2
Next Order
End Sub
```

INTERPOLAÇÃO POR SPLINES

- A interpolação linear nem sempre é a melhor forma de interpolação.
- Quando isso acontece, a solução é a utilização de polinônios de baixa ordem (primeira, segunda ou terceira ordem) para subgrupos dos dados. O polinônios que contectam os pontos conhecidos são chamados de funções splines.

SPLINES LINEARES

- A forma mais fácil de contectar dois pontos é por meio de linhas retas.
- Splines de primeira ordem para um grupo ordenado de pontos pode ser definidas por funções lineares:

$$
\begin{aligned}
f(x) &= f(x_0) + m_0(x - x_0) && x_0 \le x \le x_1 \\
f(x) &= f(x_1) + m_1(x - x_1) && x_1 \le x \le x_2 \\
&\vdots \\
f(x) &= f(x_{n-1}) + m_{n-1}(x - x_{n-1}) && x_{n-1} \le x \le x_n
\end{aligned}
$$

em que m_i é a inclinação das linhas que conectam os pontos:

$$m_i = \frac{f(x_{i+1}) - f(x_i)}{x_{i+1} - x_i}$$

EXEMPLO DE USO DE SPLINES DE PRIMEIRA ORDEM

Sabendo-se que:

x	f(x)
3,0	2,5
4,5	1,0
7,0	2,5
9,0	0,5

Utilize uma spline de primeira ordem para avaliar f(5,0).

$$m_2 = \frac{f(x_3) - f(x_2)}{x_3 - x_2} = \frac{2{,}5 - 1{,}0}{7{,}0 - 4{,}5} = 0{,}60$$

$$f(5{,}0) = f(4{,}5) + 0{,}6 \cdot (5{,}0 - 4{,}5) = 1{,}30$$

SPLINES CÚBICAS

- O objetivo das splines cúbicas é desenvolver um polinômio de terceira ordem para cada intervalo entre os pontos conhecidos, tal como:

$$f_i(x) = a_i x^3 + b_i x^2 + c_i x + d_i$$

- Assim para n+1 dados (i=0, 1, 2, ..., n), existem n intervalos e, consequentemente, 4n constantes desconhecidas para serem determinadas. Assim, 4n condições são necessárias para avaliar o valor dessas constantes. Elas são:
    1. Os valores das funções devem ser iguais nos pontos intermediários (2n-2 condições são geradas);
    2. A primeira e última função deve passar pelos pontos extremos (2 condições são geradas);
    3. As derivadas de primeira ordem nos pontos intermediários devem ser iguais (n-1 condições são geradas);
    4. As derivadas de segunda ordem nos pontos intermediários devem ser iguais (n-1 condições são geradas);
    5. As derivadas de segunda ordem nos pontos extremos devem ser iguais a zero (2 condições são geradas).

- Aplicando-se essas 4n condições chega-se a seguinte equação para cada spline cúbica:

$$f_i(x) = \frac{f_i''(x_{i-1})}{6(x_i - x_{i-1})}(x_i - x)^3 + \frac{f_i''(x_i)}{6(x_i - x_{i-1})}(x - x_{i-1})^3$$
$$+\left[\frac{f(x_{i-1})}{x_i - x_{i-1}} - \frac{f''(x_{i-1})(x_i - x_{i-1})}{6}\right](x_i - x) + \left[\frac{f(x_i)}{x_i - x_{i-1}} - \frac{f''(x_i)(x_i - x_{i-1}}{6}\right](x - x_{i-1})$$

- Essa equação contem dois valores desconhecidos, as derivadas de segunda-ordem. Esses valores podem ser obtidos resolvendo-se a seguinte equação:

$$(x_i - x_{i-1}) f''(x_{i-1}) + 2(x_{i+1} - x_{i-1}) f''(x_i) + (x_{i+1} - x_i) f''(x_{i+1})$$
$$= \frac{6}{x_{i+1} - x_i}[f(x_{i+1}) - f(x_i)] + \frac{6}{x_i - x_{i-1}}[f(x_{i-1}) - f(x_i)]$$

EXEMPLO DE USO DE SPLINES DE TERCEIRA ORDEM

Sabendo-se que:

x	f(x)
3,0	2,5
4,5	1,0
7,0	2,5
9,0	0,5

Utilize uma spline de terceira ordem para avaliar f(5,0).

A primeira etapa é avaliar as derivadas de segunda ordem, por meio de um conjunto de equações simultâneas:

$x_0 = 3,0$	$f(x_0) = 2,5$
$x_1 = 4,5$	$f(x_1) = 1,0$
$x_2 = 7,0$	$f(x_2) = 2,5$
$x_3 = 9,0$	$f(x_3) = 0,5$

Substituindo esses valores na equação para obtenção das derivadas de segunda ordem obtem-se:

$$(4,5 - 3,0) f''(3,0) + 2(7,0 - 3,0) f''(4,5) + (7,0 - 4,5) f''(7,0)$$
$$= \frac{6}{7,0 - 4,5}(2,5 - 1,0) + \frac{6}{4,5 - 3,0}(2,5 - 1,0)$$

e

$$(7{,}0-4{,}5)\,f''(4{,}5)+2(9{,}0-4{,}5)\,f''(7{,}0)+(9{,}0-7{,}0)\,f''(9{,}0)$$
$$=\frac{6}{9{,}0-7{,}0}(0{,}5-2{,}5)+\frac{6}{7{,}0-4{,}5}(0{,}5-2{,}5)$$

Sabe-se também que a derivada de segunda ordem nas extremidades deve ser igual a zero, assim:

$$f''(3{,}0)=0$$

e

$$f''(9{,}0)=0$$

Assim, o sistema de equações fica reduzido a:

$$8{,}0\,f''(4{,}5)+2{,}5\,f''(7)=9{,}6$$

e

$$2{,}5\,f''(4{,}5)+9{,}0\,f''(7)=-9{,}6$$

Resolvendo-se o sistema de equações obtem-se:

$$f''(4{,}5)=1{,}67909$$

e

$$f''(7{,}0)=-1{,}53308$$

Assim a equação da spline para $3{,}0\le x\le 4{,}5$ é dada por:

$$f_1(x)=\frac{1{,}67909}{6(4{,}5-3{,}0)}(x-3{,}0)^3+\frac{2{,}5}{4{,}5-3{,}0}(4{,}5-x)$$
$$+\left[\frac{1{,}0}{4{,}5-3{,}0}-\frac{1{,}67909(4{,}5-3{,}0)}{6}\right](x-3)$$

ou seja,

$$f_1(x)=0{,}186566(x-3{,}0)^3+1{,}666667(4{,}5-x)+0{,}246894(x-3{,}0)$$

De forma similar obtem-se para o segundo e o terceiro intervalos.

Para $4{,}5 \le x \le 7{,}0$

$$f_2(x) = 0{,}111939(7{,}0 - x)^3 - 0{,}102205(x - 4{,}5)^3 - 0{,}299621(7{,}0 - x) + 1{,}638783(x - 4{,}5)$$

e

Para $7{,}0 \le x \le 9{,}0$

$$f_3(x) = -0{,}127757(9{,}0 - x)^3 + 1{,}761027(9{,}0 - x) + 0{,}25(x - 7{,}0)$$

Para estimar o valor f(x = 5,0), tem-se

$$f_2(5{,}0) = 0{,}111939(7{,}0 - 5{,}0)^3 - 0{,}102205(5{,}0 - 4{,}5)^3 - 0{,}299621(7{,}0 - 5{,}0) + 1{,}638783(5{,}0 - 4{,}5) = 1{,}102886$$

ALGORITMO DE COMPUTADOR PARA IMPLEMENTAÇÃO DE SPLINES CÚBICAS

```
SUBROUTINE Spline(x,y,n,xu,yu,dy,d2y)
LOCAL e_n,f_n,g_n,r_n,d2x_n
CALL Tridiag(x,y,n,e,f,g,r)
CALL LinSol(e,f,g,r,n-1,d2x)
CALL Interpol(x,y,n,d2x,xu,yu,dy,d2y)
END Spline


SOUBROUTINE Tridiag(x,y,n,e,f,g,r)
f_1=2*(x_2-x_0)
g_1=(x_2-x_1)
r_1=6/(x_2-x_1)*(y_2-y_1)
r_1=r_1+6/(x_1-x_0)*(y_0-y_1)
DO i=2,n-2
    e_i=(x_i-x_i-1)
    f_i=2*(x_i+1-x_i-1)
    g_i=(x_i+1-x_i)
    r_i=6/(x_i+1-x_i)*(y_i+1-y_i)
    r_i=r_i+6/(x_i-x_i-1)*(y_i-1-y_i)
END DO
e_n-1=(x_n-1-x_n-2)
f_n-1=2*(x_n-x_n-2)
```

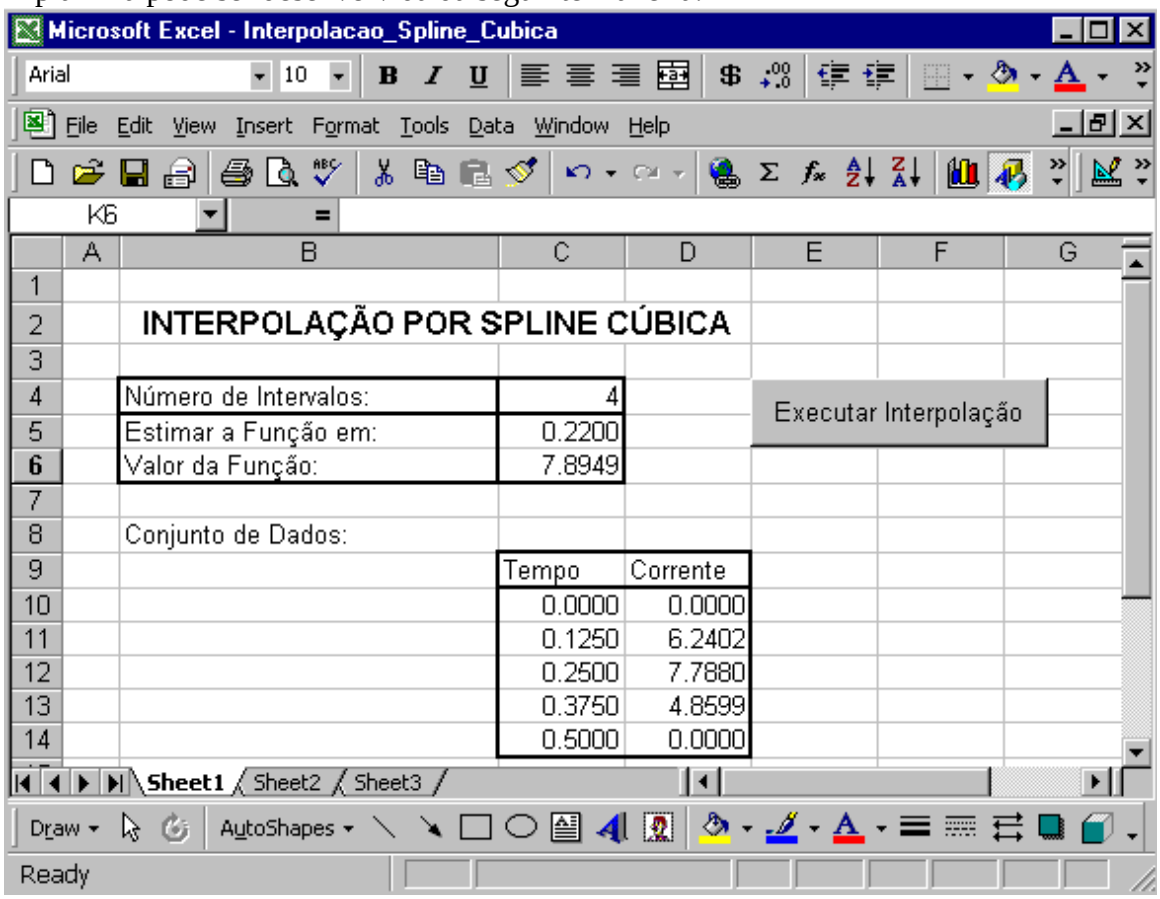
```
r_{n-1}=6/(x_n-x_{n-1})*(y_n-y_{n-1})
r_{n-1}=r_{n-1}+6/(x_{n-1}-x_{n-2})*(y_{n-2}-y_{n-1})
END Tridiag


SUBROUTINE Interpol(x,y,n,d2x,xu,yu,dy,d2y)
flag=0
i=1
DO
   IF xu≥x_{i-1} AND xu≤x_i THEN
      c1 = d2x_{i-1}/6/(x_i-x_{i-1})
      c2 = d2x_i/6/(x_i-x_{i-1})
      c3 = (y_{i-1}/(x_i-x_{i-1}))-d2x_{i-1}*(x_i-x_{i-1})/6
      c4 = (y_i/(x_i-x_{i-1}))-d2x_i*(x_i-x_{i-1})/6
      t1 = c1*(x_i-xu)^3
      t2 = c2*(xu-x_{i-1})^3
      t3 = c3*(x_i-xu)
      t4 = c4*(xu-x_{i-1})
      yu = t1 + t2 + t3 + t4
       t1= -3*c1*(x_i-xu)^2
       t2 = 3*c2*(xu-x_{i-1})^2
       t3 = -c3
       t4 = c4
       dy = t1 + t2 + t3 + t4
       t1 = 6*c1*(x_i-xu)
       t2 = 6*c2*(xu-x_{i-1})
       d2y = t1 + t2
       flag = 1
    ELSE
       i = i+1
    END IF
    IF i = n+1 OR flag = 1 EXIT
END DO
IF flag = 0 THEN
    PRINT "Fora da faixa"
    Pause
END IF
END Interpol
```

UNIVERSIDADE FEDERAL DE VIÇOSA
DEPARTAMENTO DE ENGENHARIA AGRÍCOLA
ENG590 - Métodos Numéricos Aplicados à Engenharia

DESENVOLVENDO UMA PLANILHA EXCEL PARA INTERPOLAÇÃO USANDO SPLINE CÚBICA

A planilha pode ser desenvolvida da seguinte maneira:



O código em VBA Aplication pode ser escrito assim:

```
Private Sub CommandButton1_Click()
   Call Entrada_de_Dados4
End Sub


Private Sub Entrada_de_Dados4()
   Dim n As Integer
   Dim x(100) As Double
   Dim y(100) As Double
   Dim xu As Double
   Dim yu As Double
   Dim dy As Double
   Dim d2y As Double
   Dim ilin As Integer
   Dim icol As Integer
```

```
  n = Sheet1.Cells(4, 3)
  xu = Sheet1.Cells(5, 3)
  For ilin = 10 To 10 + n
    x(ilin - 10) = Sheet1.Cells(ilin, 3)
    y(ilin - 10) = Sheet1.Cells(ilin, 4)
  Next ilin
  Call Spline(x, y, n, xu, yu, dy, d2y)
  Sheet1.Cells(6, 3) = yu
End Sub


Private Sub Spline(x, y, n, xu, yu, dy, d2y)
Dim e(100) As Double
Dim f(100) As Double
Dim g(100) As Double
Dim r(100) As Double
Dim d2x(100) As Double
Dim nless1 As Integer
nless1 = n - 1
Call Tridiag(x, y, n, e, f, g, r)
Call LinSol(e, f, g, r, nless1, d2x)
Call Interpol(x, y, n, d2x, xu, yu, dy, d2y)
End Sub


Private Sub Tridiag(x, y, n, e, f, g, r)
Dim i As Integer
f(1) = 2 * (x(2) - x(0))
g(1) = (x(2) - x(1))
r(1) = 6 / (x(2) - x(1)) * (y(2) - y(1))
r(1) = r(1) + 6 / (x(1) - x(0)) * (y(0) - y(1))
If ((n - 2) >= 2) Then
  For i = 2 To n - 2
     e(i) = (x(i) - x(i - 1))
     f(i) = 2 * (x(i + 1) - x(i - 1))
     g(i) = (x(i + 1) - x(i))
     r(i) = 6 / (x(i + 1) - x(i)) * (y(i + 1) - y(i))
     r(i) = r(i) + 6 / (x(i) - x(i - 1)) * (y(i - 1) - y(i))
  Next i
End If
e(n - 1) = (x(n - 1) - x(n - 2))
f(n - 1) = 2 * (x(n) - x(n - 2))
r(n - 1) = 6 / (x(n) - x(n - 1)) * (y(n) - y(n - 1))
r(n - 1) = r(n - 1) + 6 / (x(n - 1) - x(n - 2)) * (y(n - 2) - y(n - 1))
End Sub
```

```
Private Sub LinSol(e, f, g, r, nless1, d2x)
Dim k As Integer
For k = 2 To nless1
      e(k) = e(k) / f(k - 1)
      f(k) = f(k) - e(k) * g(k - 1)
Next k
For k = 2 To nless1
      r(k) = r(k) - e(k) * r(k - 1)
Next k
   d2x(nless1) = r(nless1) / f(nless1)
   'Stop
For k = nless1 - 1 To 1 Step -1
      d2x(k) = (r(k) - g(k) * d2x(k + 1)) / f(k)
Next k
End Sub


Private Sub Interpol(x, y, n, d2x, xu, yu, dy, d2y)
Dim flag As Integer
Dim i As Integer
Dim c1 As Double
Dim c2 As Double
Dim c3 As Double
Dim c4 As Double
Dim t1 As Double
Dim t2 As Double
Dim t3 As Double
Dim t4 As Double
flag = 0
For i = 1 To n
  If ((xu >= x(i - 1)) And (xu < x(i)) And (flag = 0)) Then
     c1 = d2x(i - 1) / 6 / (x(i) - x(i - 1))
     c2 = d2x(i) / 6 / (x(i) - x(i - 1))
     c3 = (y(i - 1) / (x(i) - x(i - 1))) - d2x(i - 1) * (x(i) - x(i - 1)) / 6
     c4 = (y(i) / (x(i) - x(i - 1))) - d2x(i) * (x(i) - x(i - 1)) / 6
     t1 = c1 * (x(i) - xu) * (x(i) - xu) * (x(i) - xu)
     t2 = c2 * (xu - x(i - 1)) * (xu - x(i - 1)) * (xu - x(i - 1))
     t3 = c3 * (x(i) - xu)
     t4 = c4 * (xu - x(i - 1))
     yu = t1 + t2 + t3 + t4
     t1 = -3 * c1 * (x(i) - xu) * (x(i) - xu)
     t2 = 3 * c2 * (xu - x(i - 1)) * (xu - x(i - 1))
     t3 = -c3
     t4 = c4
     dy = t1 + t2 + t3 + t4
     t1 = 6 * c1 * (x(i) - xu)
     t2 = 6 * c2 * (xu - x(i - 1))
     d2y = t1 + t2
     flag = 1
```

```
  End If
Next i
End Sub
```